

Data

The dataset contains 1462 survey responses about four cities — Dubai, Rio de Janeiro, New York City, and Paris — collected from ten questions (Q1 – Q10) and response id . Each response is labelled with the city it was actually describing (Label), which is the target we predict.

01

Missing values

Before any exploration, we checked for missing data. Q10 was missing in 43 of 1462 rows (2.94%). Six other columns (Q2, Q3, Q5, Q7, Q8, Q9) were each missing exactly one value, affecting four distinct rows total.

COLUMN	MISSING	%
Q10	43	2.94%
Q2 / Q3 / Q5 / Q7 / Q8 / Q9	1 each	0.07% each

Decision: the four rows missing a single-value field were dropped — losing under 0.3% of the data was preferable to imputing across six unrelated questions in the same rows. Missing Q10 was filled with an empty string rather than dropped.

02

Feature distributions

Q1–Q4 are ratings for popularity, virality, architectural uniqueness, street-party association; Q7–Q9 are numeric questions (temperature guess, height guess, distance guess); Q5 is a select list of travel companions.

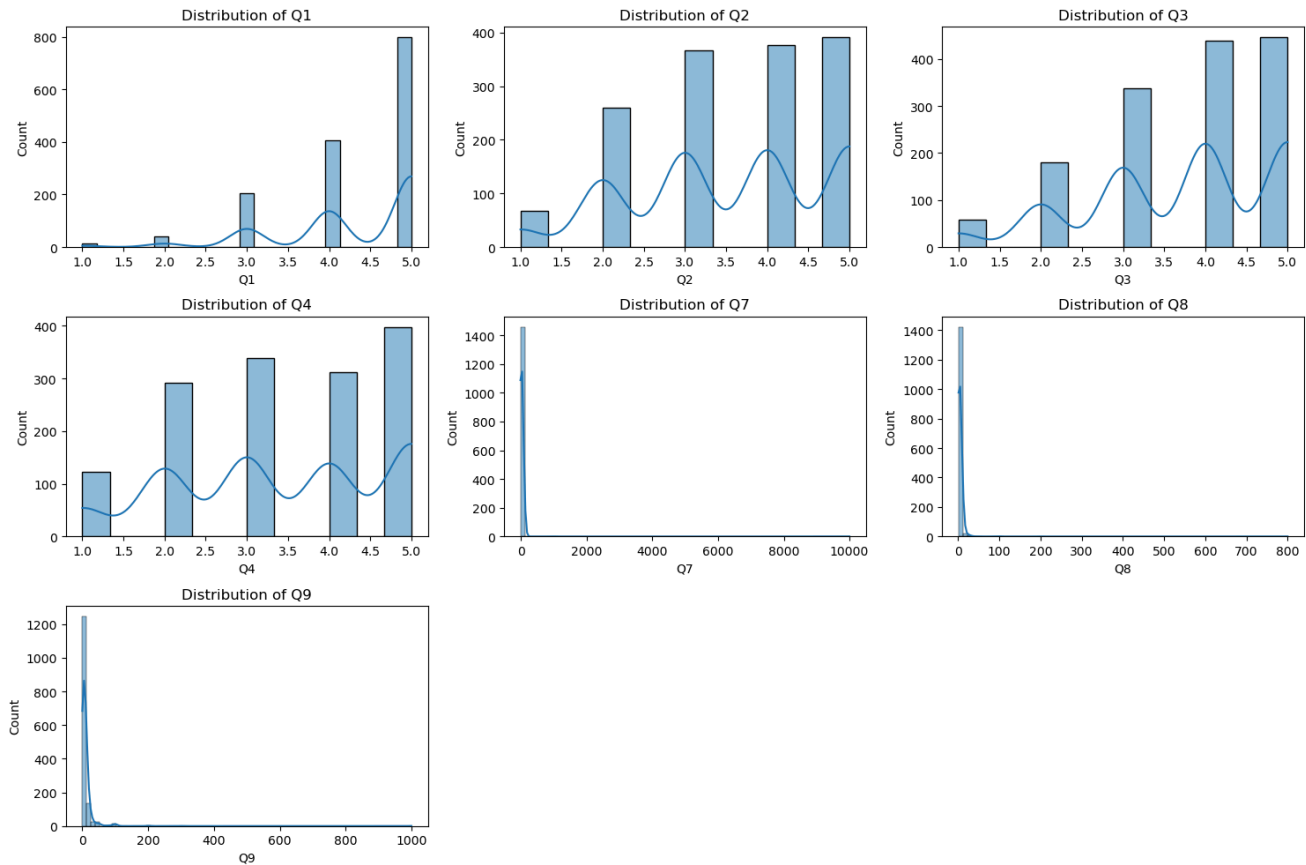


Figure 1. Distributions of the six numeric questions (before cleaning).

Q1–Q4 are all left-skewed with higher ratings — nearly no responses rate a city below 3, and Q1 (popularity) is very high, with over half the responses at 5. Q7– Q9 was very noisy: many values overwhelmingly stayed around zero, but the distribution has a long tail stretching to 10,000 (Q7) and 800–1000 (Q8, Q9) — these outliers will hide the real shape of the distribution near zero, which is a problem we address later.

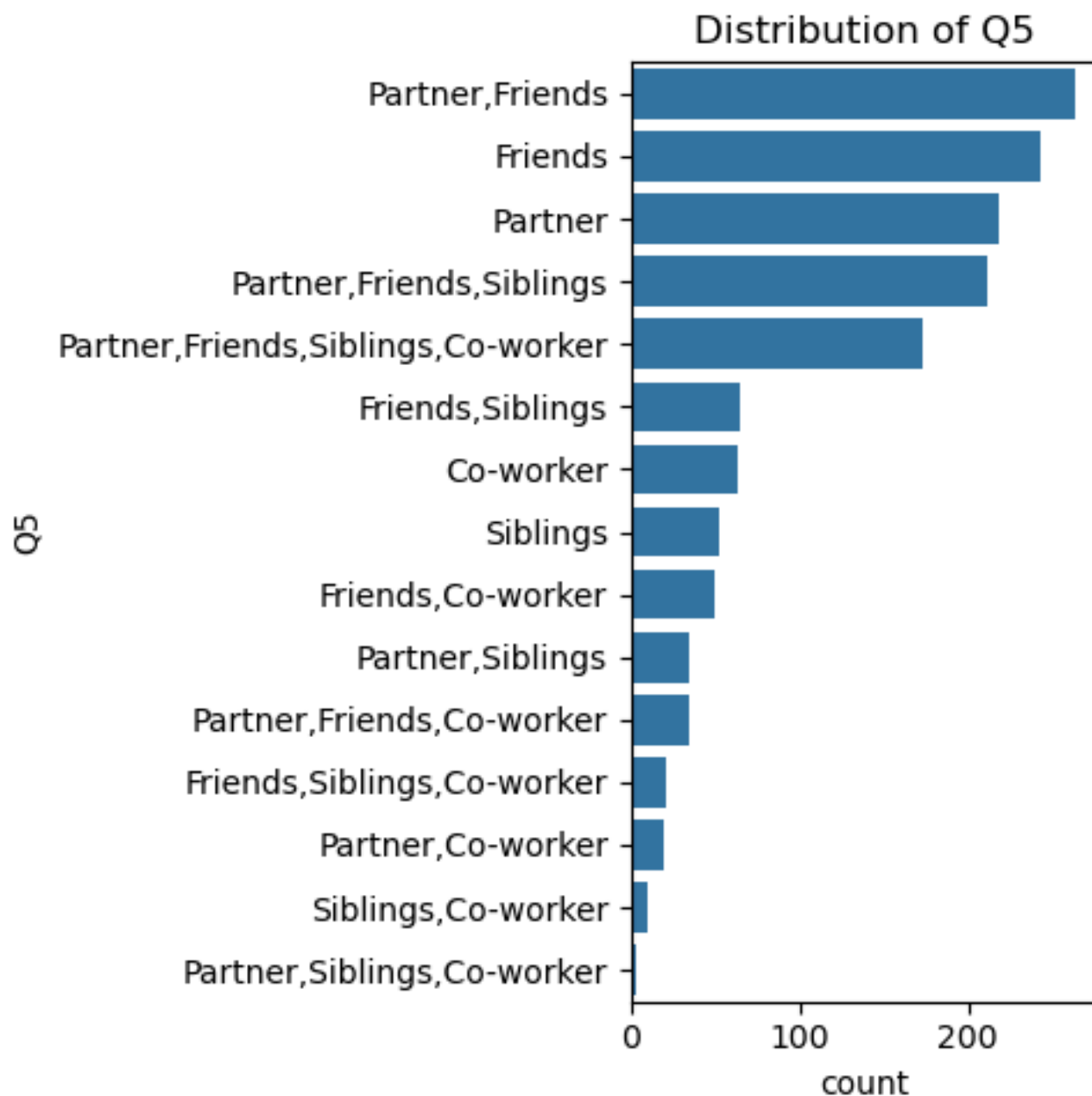


Figure 2. Frequency of each distinct combination of travel companions reported in Q5.

Reading it: "Partner, Friends" and "Friends" are the two most common answers, as well as "Partner" and "Partner, Friends, Siblings, Co-worker" make up majority of the responses. . Because Q5 is multi-select, Instead of encoding this as separate 14 categories/features and we interpreted it as 4 indicator features.

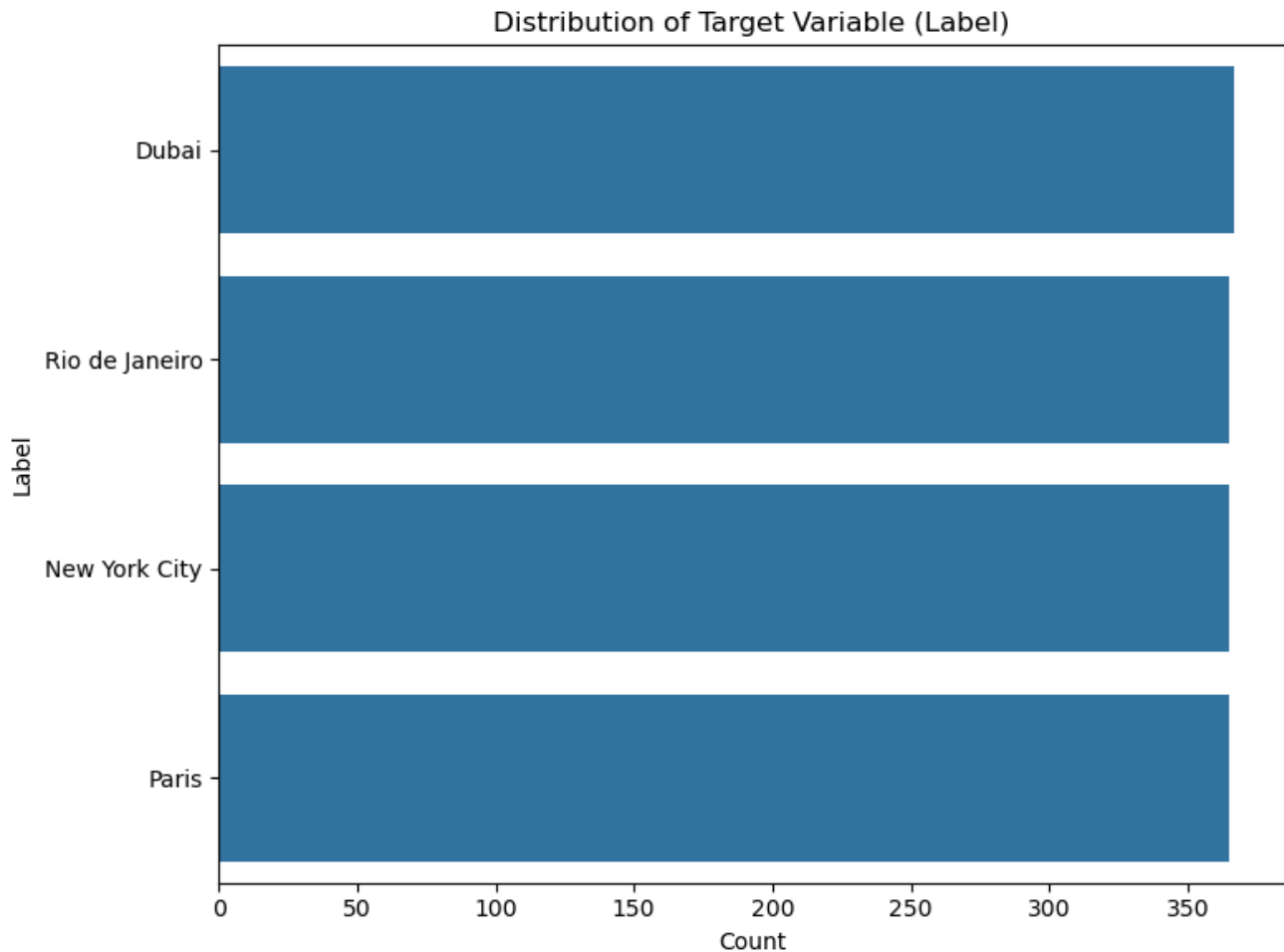


Figure 3. Count of responses per city (the target variable).

Reading it: the four classes are almost perfectly balanced (~365 responses each). This matters directly for the rest of the project since it tells us that accuracy is a fair evaluation metric since there doesn't exist any class imbalance.

03

Feature–target relationships

Before deciding which features to keep, we checked (a) whether the numeric features are redundant with each other, and (b) whether each feature actually affects the four cities.

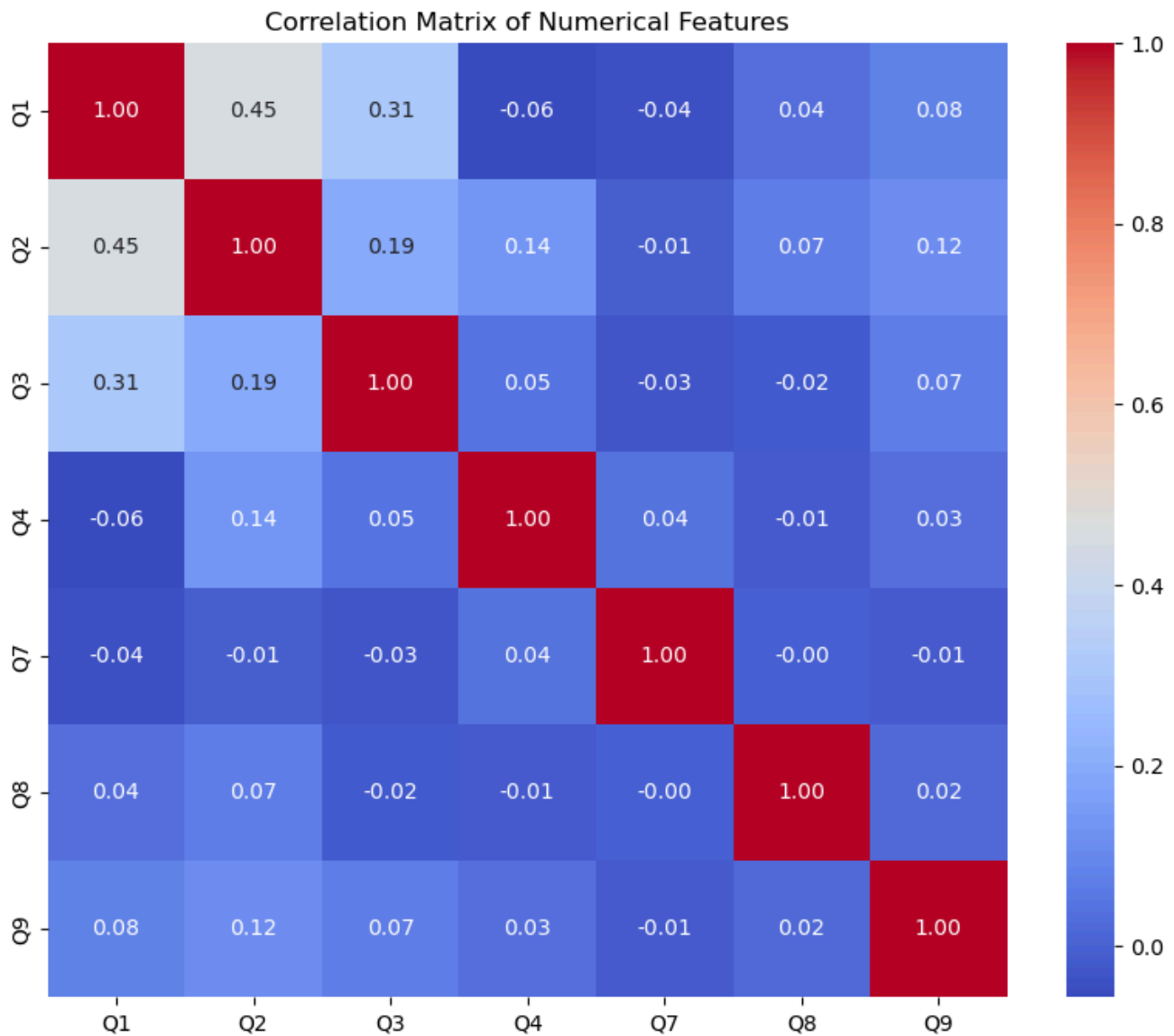


Figure 4. Pairwise correlation between the numeric questions.

Reading it: Q1, Q2, and Q3 (popularity, virality, architectural uniqueness) are somewhat correlated with each other ($r = 0.31\text{--}0.45$), since a more popular city is likely rated more viral and distinctive too — but not strongly enough to call any of them redundant. Q4 and Q7–Q9 are essentially uncorrelated with everything ($|r| < 0.15$), meaning these features provides a lot of independent information.

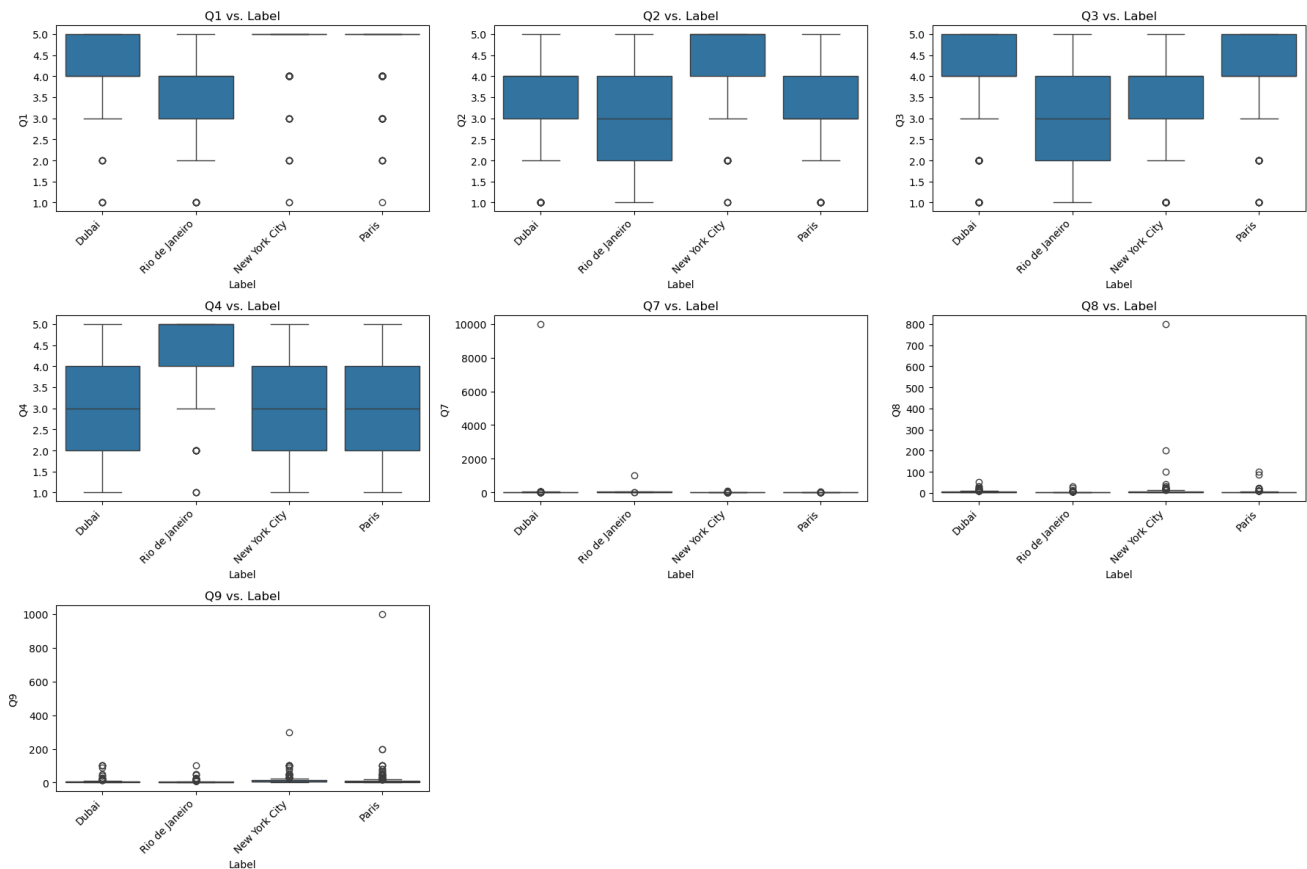


Figure 5. Q1–Q4 and Q7–Q9 grouped by true city label.

Reading it: Q1–Q4 responses all shift visibly depending on city — Rio has a noticeably lower median for Q1 (popularity) and Q3 (architecture) but the highest Q4 (street-party association, consistent with Carnival), while NYC skews high on Q2 (virality). Q7–Q9, However, show almost no separation by city after the outliers are accounted for. This is because the boxes and whiskers sit on top of each other across all four cities, telling us these three questions themselves give us very little information on the label.

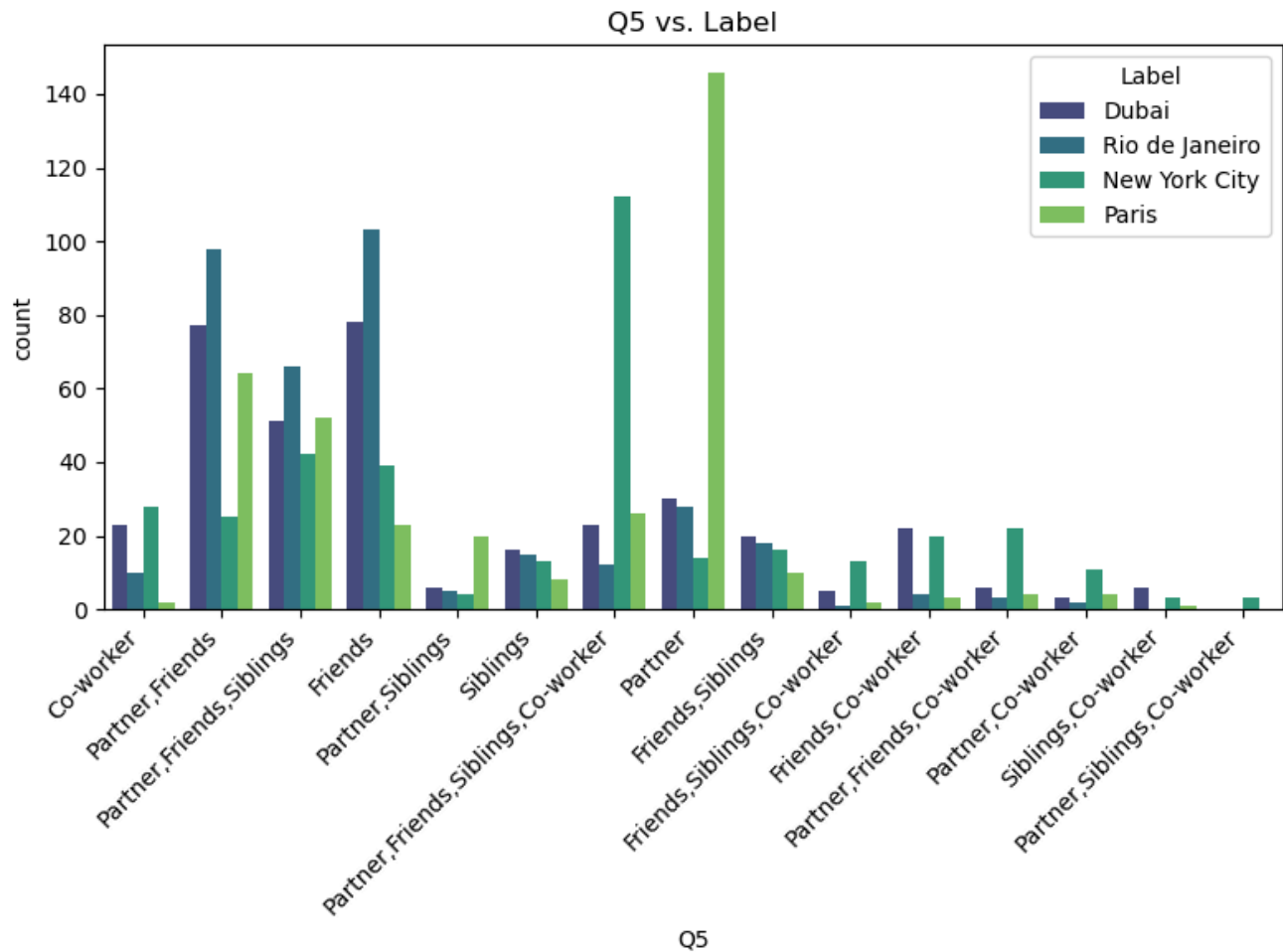


Figure 6. Q5 responses grouped by true city label.

Reading it: companion responses are different meaningfully by city. "Friends" is not a common response for Paris, "Partner, Friends, Siblings, Co-worker" is the most common combination for NYC, and "Partner" is very common toward Paris and NYC over Dubai and Rio. This is why we decomposed Q5 into binary since it's the individual selections, not the original multiple-selection, that have a signal.

04

Q6 representation:

Q6 asks respondents to rank six words (Skyscrapers, Sport, Art and Music, Carnival, Cuisine, Economic) from 1–6 by association with the city. We parsed the raw

"word=>rank, ..." string into six separate numeric columns, one per word.

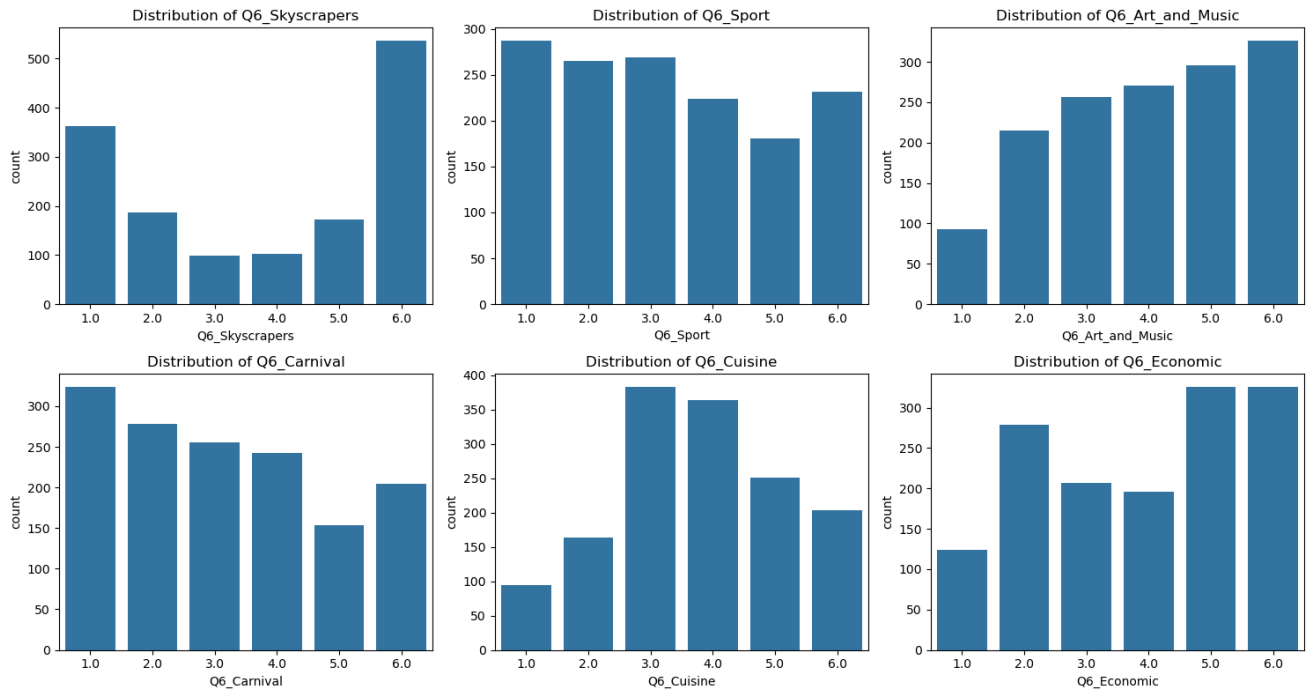


Figure 7. Distribution of the rank given to each Q6 word, across all responses.

Reading it: It is clear that these responses are not uniformly distributed — Skyscrapers is either heavily ranked either 1 or 6, while Cuisine is often ranked in the middle (3–4). This is evidence that word rank correlates with something.

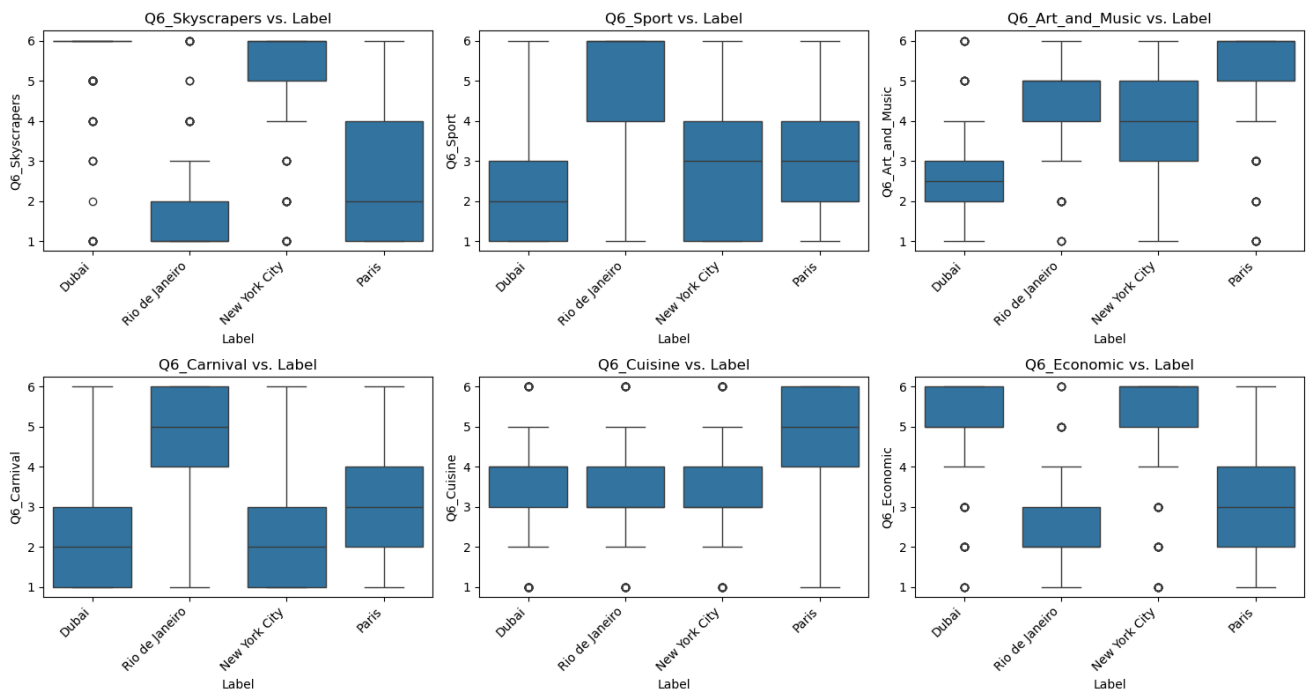


Figure 8. Each Q6 word's rank, grouped by true city label.

Reading it: There is a clear relation between label and these features shown in these box plots. Dubai and NYC often rank Skyscrapers high while Rio and Paris rank it low; Rio ranks Carnival and Sport far above the other three cities; Paris is the only city ranked high on Cuisine; and Dubai/NYC are ranked high on Economic. These relations justify why each of these words were made into separate columns.

05

Outliers in Q7–Q9

Q7–Q9 are numeric questions and some of the responses were very extreme. We used the IQR rule to indicate outliers before deciding how to handle it.

QUESTION	IQR FENCE	# OUTLIERS	OBSERVED RANGE
Q7	(−25.0, 55.0)	3	−15 to 10,000
Q8	(−2.5, 9.5)	103	1 to 800
Q9	(−7.5, 20.5)	91	0 to 1,000

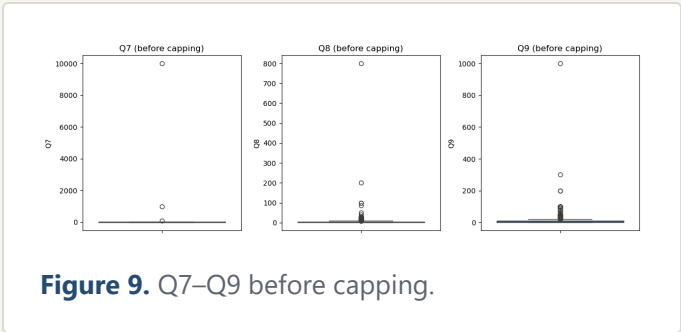


Figure 9. Q7–Q9 before capping.

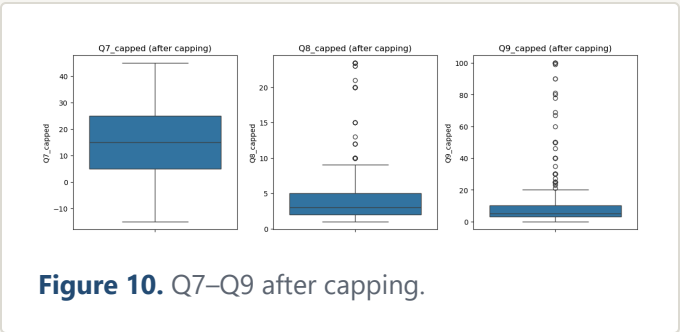


Figure 10. Q7–Q9 after capping.

Reading it: before capping, a single Q7 response of 10,000 stretched the range of values so far that the other 1,460 points seem negligible (Figure 9) — the same thing happens for Q8 and Q9. **Decision:** rather than dropping 91–103 rows per question (which would be a large chunk of the dataset), we capped Q7 at a fixed range `[-20, 45]` and capped Q8/Q9 at their 99th percentile. Figure 10 shows the same three questions afterward, now with an interpretable box plot instead of a flat line at the bottom of the chart. These bounds are computed from `train_df` **only** after the train/val/test split (Section 8), since computing them before splitting would let val/test would be a data leakage.

06

Q10 representation

Q10 was an open-ended quote about the city. We decided to reduce this feature to two numeric signals: word count, and four keywords (`tallest` , `carnival` , `eiffel` , `skyscraper`) chosen by inspecting the most frequent words per city.

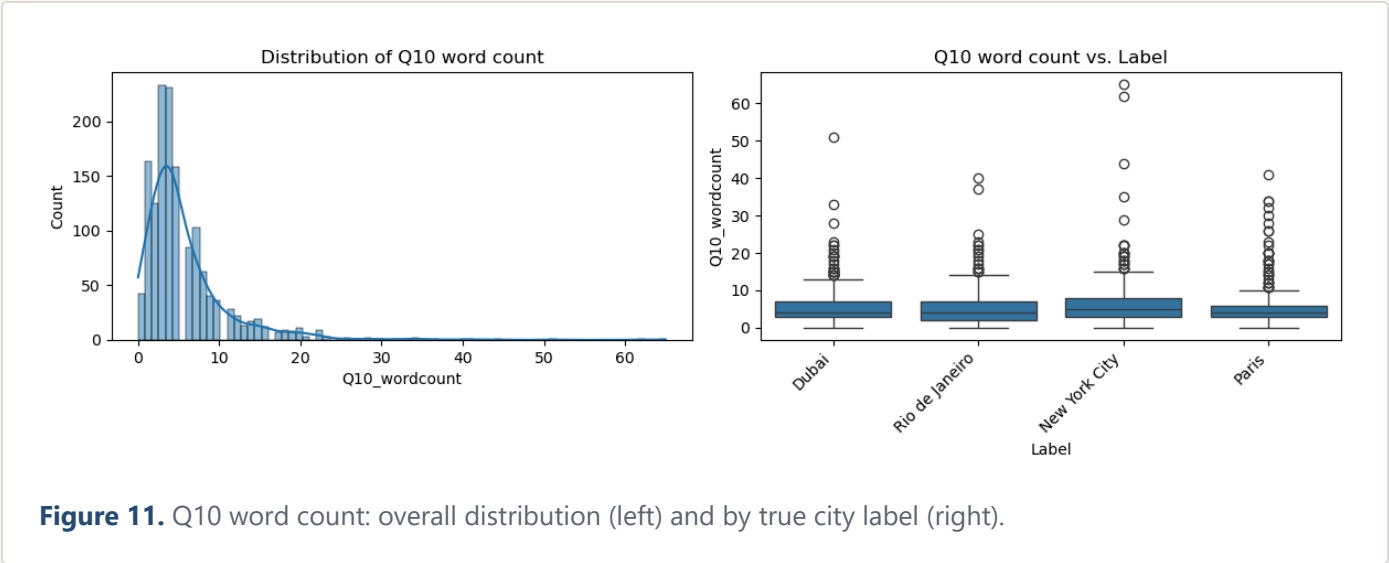


Figure 11. Q10 word count: overall distribution (left) and by true city label (right).

Reading it: word count is heavily right-skewed (most quotes are 2–8 words) and nearly identical across all four cities — the boxes in the right panel are all almost the same. The four keyword features were rare (15–27 rows each out of 1458) but were veru city-specific when they did appear — e.g. "carnival" only appears in Rio quotes — which is why they were kept despite their sparsity.

07

Final feature representations

Putting all the above together, each response became a 22-dimensional numeric feature vector:

SOURCE	COLUMNS	TYPE
Q1–Q4	Q1, Q2, Q3, Q4	1–5 rating
Q5 (companions)	Q5_Partner, Q5_Friends, Q5_Siblings, Q5_Co-worker	binary indicator
Q6 (word ranks)	Q6_Skyscrapers, Q6_Sport, Q6_Art_and_Music, Q6_Carnival, Q6_Cuisine, Q6_Economic	rank, 1–6
Q7–Q9	Q7_capped, Q8_capped, Q9_capped	capped numeric

Q10 (text)

Q10_wordcount , 4× Q10_kw_*

count /
binary

Why nothing was dropped: Q7–Although Q9 and the Q10 features showed the weakest visual signals by city (Figures 5, 11), but we kept them rather than removing them for ease — each is cheap to compute and none showed evidence of actively hurting a model in our exploration and a feature that looks weak marginally can still contribute in combination with others. `id` is excluded from the feature vector entirely — its only role is defining the split below.

08

Train / validation / test split

Most respondents answered about more than one city (only 367 unique `id`s across 1458 remaining rows), so a random row-level split would let the same person's rating appear in both train and test this is a problem because the model could then learn to recognize *respondents* rather than *cities*. To address this, we used **group-based splitting** (based on `id`) instead, guaranteeing no respondent's rows appear in more than one split.

1017

TRAIN (70%)

218

VALIDATION (15%)

223

TEST (15%)

Class balance still held within every split (24.7%–25.2% per city in each of train/val/test), so the group constraint doesn't introduce any class imbalances. For hyperparameter selection specifically, we additionally ran 5-fold **GroupKFold** cross-validation *within the training set* (still grouped by `id`), because a single 218-row validation set is noisy enough that its "best" hyperparameter can differ from what 5-fold CV agrees on, which we confirmed while tuning our models. The validation set is then used once, after hyperparameters are fixed, to get an unbiased comparison accuracy number across different model families; the test set remained unseen.

Model

Every model below is trained and evaluated on the same 22-dimensional feature vector defined in the Data section, using the same group-based

train/validation/test split. We explored five model families in total.

01

Setup

(1) hyperparameters for every model were selected using 5-fold `GroupKFold` cross-validation on the training set, grouped by response `id` — to address the same leakage concern as the original split, since the model could learn response patterns instead of the city label patterns if their rows were split randomly; and (2) the validation set is used only once per family (after its hyperparameters are already optimized by cross-validation) to produce the accuracy number for model selection. **We chose to experiment with the following model families:**

FAMILY	SKLEARN CLASS
Decision Tree	<code>DecisionTreeClassifier</code>
Random Forest	<code>RandomForestClassifier</code>
Gradient Boosting	<code>GradientBoostingClassifier</code>
Neural Network (MLP)	<code>MLPClassifier</code>
Softmax Regression	<code>LogisticRegression</code>

02

Decision Tree

The input is the 22-feature vector with no additional preprocessing aside from the transformations mentioned earlier. This is because a tree model makes no assumption about feature scale or column type since it just splits based on thresholds, regardless of whether a column is numeric, one-hot encoded vector etc. which is good because this dataset has multiple feature types and they are on different scales.

An unrestricted tree fit the training data perfectly (100% train accuracy) but generalized poorly (~80% validation accuracy) indicating overfitting. We proceeded to address this using regularization methods such as pruning (`max_depth` , `min_samples_leaf`). We also looked to reduce variance using ensembles of trees instead of a single one.

Random Forest & Gradient Boosting

Again we use the 22-feature representation and need no scaling, for the same reason as the single tree. The only real change in how they're applied is *which* trees get combined and how: Random Forest bags many trees trained on bootstrapped samples and averages their votes, while Gradient Boosting fits trees sequentially, each one correcting the previous ensemble's residual errors. We reached for both immediately after the single tree specifically because its failure mode was high variance, not a wrong representation — ensembling is the direct fix for variance, so it made more sense to try before reaching for a different model family entirely.

Results: both delivered on that expectation. Under the same 5-fold `GroupKFold` cross-validation, Random Forest reached **89.1% mean accuracy** (std 0.022) and Gradient Boosting reached **89.6%** (std 0.021) — both a clear jump over the single tree's 81.8% (std 0.046), and just as tellingly, the fold-to-fold standard deviation roughly halved for both. That's the actual signature of variance reduction, not just a lucky accuracy bump: the single tree's problem was inconsistency across folds, and both ensembles fixed exactly that. Gradient Boosting edged out Random Forest by half a point and became the strongest tree-based model; refit on the full training set and checked once against validation, it reached **93.1% validation accuracy** (against 100% training accuracy — it still fits the training set perfectly, but generalizes far better than the unpruned single tree did). Its feature importances were dominated by `Q7_capped` (0.30) and `Q6_Skyscrapers` (0.14) — the same two features the single tree also leaned on most heavily.

04

Neural Network (MLP, softmax output)

Applied via `MLPClassifier`, which already uses a softmax output layer with cross-entropy loss for a multi-class target like ours — no manual configuration needed for that part. The real application decision here is preprocessing: unlike the tree family, an MLP's hidden-layer computations are scale-sensitive, so every feature had to be standardized first. To do this correctly under cross-validation, the `StandardScaler` is fit fresh inside each CV fold, using only that fold's training rows, rather than once on the whole training set beforehand — fitting it once first would let each fold's held-out rows quietly influence the scaling statistics used to preprocess them, the same class of leak we'd already caught and fixed for the Q7–Q9 capping bounds in the Data section.

We tried an MLP specifically to test whether letting the model learn arbitrary nonlinear combinations of features — rather than the axis-aligned, one-feature-at-a-time splits a tree uses — would pay off on this data.

Implementation note: `MLPClassifier(early_stopping=True)` crashes on string class labels in this sklearn version — its internal validation-score check calls `np.isnan()` on the predicted city names, which aren't numeric. We left `early_stopping` off; the outer cross-validation plus the single external validation check already guard against overfitting without it.

05

Softmax (Multinomial Logistic) Regression

Applied via `LogisticRegression`, which fits one true multinomial softmax model directly — a single linear score per city, normalized by softmax — rather than training four independent one-vs-rest binary classifiers and picking whichever is most confident. It needs the same per-fold standardization as the MLP, for the same reason.

This was deliberately the last family we tried, as a linear baseline: if a plain linear decision boundary could match the trees and the MLP, that would be strong evidence that most of the city signal in this data is already close to linearly separable (consistent with what Figures 4–5 in the Data section suggested for features like `Q4` and `Q2`), and that the added complexity of the other families isn't earning its keep. It also happens to be the cheapest model to reimplement from scratch in plain numpy, which matters directly for the final `pred.py` submission (Model Choice & Hyperparameters).

Model families and application choices reproduced from `csc311_predictor_fixed.ipynb`. Hyperparameter search evidence and the final cross-model comparison are covered separately in Model Choice & Hyperparameters.

